01 — Exploratory Data Analysis

Daily revenue from an independent community pharmacy, 15 December 2019 to 10 May 2021: 470 observed days across a 513-day calendar window (91.6% coverage). The pharmacy traded on every day in the window; the 43 missing days reflect export failures rather than closures.

What the series measures. The series records cash received at the counter each day, approximately 65% of the pharmacy's total revenue: cash purchases, patient co-payments on insured prescriptions (0–20% of the price, depending on coverage), and non-covered products such as vitamins and para-pharmaceuticals. The remaining 35%, the government-reimbursed share of insured prescriptions, goes through a separate claims system and is not included, so all findings describe this 65%. Fully covered prescriptions (typically for chronic conditions) contribute nothing to the visible series, so the series over-represents discretionary and acute purchases; declines during the pandemic may therefore overstate the fall in total pharmacy business.

Credit accounts. Some regular customers (families and individuals) buy on account and pay later. These small, frequent settlements total roughly 2–3% of total revenue over the sample; they are recorded on the day the customer pays rather than the day of purchase and are left in the series. Institutional customers occasionally settle large credit accounts in a single payment; some years bring one or two such settlements, others none. The only such settlement in this sample, on 25 December 2019 (about a year of purchases paid at once), is excluded from level comparisons and from the models in NB02–NB05: with a single occurrence it cannot be modelled, only set aside.

I ran the pharmacy throughout this period. Where individual days are attributed to specific causes, those attributions come from the business's records and my own knowledge, not from data-derived inference.

For confidentiality, all revenue values are multiplied by a single positive constant, chosen so that the sample mean is 120,000. Ratios, percentage changes, break dates, autocorrelations, test statistics and p-values are unaffected by this transformation; levels reported in LC (scaled local currency), including the mean, standard deviation, minimum, maximum, and any sum in currency units, are not the pharmacy's actual takings.

This notebook is a first read of the data — coverage, distribution, weekly and monthly structure, and the pattern of missing days — before hypothesis testing in NB02, NB03 and NB04, and the forecasting comparison in NB05.

In [1]:
```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from matplotlib.patches import Patch
from scipy.stats import skew, kurtosis
from IPython.display import display

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (11, 4)
plt.rcParams["axes.titleweight"] = "bold"
pd.set_option("display.max_colwidth", None)
pd.set_option("display.float_format", "{:,.0f}".format)

DATA = Path("../data/daily_sales_final.csv")
df = pd.read_csv(DATA, parse_dates=["date"])
df = df.sort_values("date").reset_index(drop=True)
print(df.shape)
df.head()
```

```
(470, 2)
```

Out[1]:

	date	revenue_scaled
0	2019-12-15	141,552
1	2019-12-16	131,409
2	2019-12-17	118,912
3	2019-12-18	154,326
4	2019-12-19	151,141

1. Coverage

In [2]:

```
min_d, max_d = df["date"].min(), df["date"].max()
expected = pd.date_range(min_d, max_d, freq="D")
missing = sorted(set(expected) - set(df["date"]))

print(f"Date range: {min_d.date()} to {max_d.date()}")
print(
    f"Expected days: {len(expected)}   Actual days: {len(df)}   "
    f"Missing: {len(missing)} ({100 * len(missing) / len(expected):.1
f}%)"
)

if missing:
    ranges = []
    start = prev = missing[0]
    for d in missing[1:]:
        if (d - prev).days == 1:
            prev = d
        else:
            ranges.append((start, prev))
            start = prev = d
    ranges.append((start, prev))
    print(f"\nMissing-day ranges ({len(ranges)}):")
    for s, e in ranges:
        n = (e - s).days + 1
```

```
        print(f"  {s.date()} to {e.date()} ({n} day{'s' if n > 1 else ''})")
```

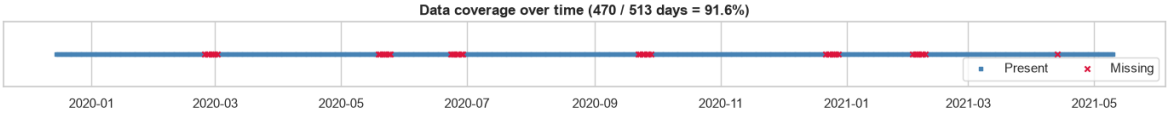
```
Date range: 2019-12-15 to 2021-05-10
Expected days: 513   Actual days: 470   Missing: 43 (8.4%)

Missing-day ranges (7):
  2020-02-25 to 2020-03-02 (7 days)
  2020-05-19 to 2020-05-25 (7 days)
  2020-06-23 to 2020-06-29 (7 days)
  2020-09-22 to 2020-09-28 (7 days)
  2020-12-22 to 2020-12-28 (7 days)
  2021-02-02 to 2021-02-08 (7 days)
  2021-04-13 to 2021-04-13 (1 day)
```

43 of 513 days (8.4%) have no data. The pharmacy traded on every day of the period; the gaps are failures of the daily sales export: six full weeks, each running Tuesday–Monday, and one further day on 13 April 2021. I treat them as missing completely at random, which cannot be verified from the data. Because each missing week removes one of every weekday, the six weekly gaps do not bias day-of-week comparisons; the single missing Tuesday leaves only a minor residual asymmetry on one weekday. Missing days are left as gaps and are not imputed.

In [3]:
```
cov = pd.DataFrame({"date": expected})
cov["present"] = cov["date"].isin(df["date"])

fig, ax = plt.subplots(figsize=(14, 1.6))
ax.scatter(
    cov[cov["present"]]["date"],
    [1] * cov["present"].sum(),
    s=6,
    c="steelblue",
    marker="s",
    label="Present",
)
ax.scatter(
    cov[~cov["present"]]["date"],
    [1] * (~cov["present"]).sum(),
    s=20,
    c="crimson",
    marker="x",
    label="Missing",
)
ax.set_yticks([])
ax.set_title(
    f"Data coverage over time ({len(df)} / {len(expected)} days "
    f"= {100 * len(df) / len(expected):.1f}%)"
)
ax.legend(loc="lower right", ncol=2)
plt.tight_layout()
plt.savefig("../charts/nb01_coverage.png", dpi=120, bbox_inches="tight")
plt.show()
```



2. Distribution and summary statistics

In [4]:
```
summary = df["revenue_scaled"].describe(percentiles=[0.05, 0.25, 0.5, 0.75, 0.95])
print(summary)
```

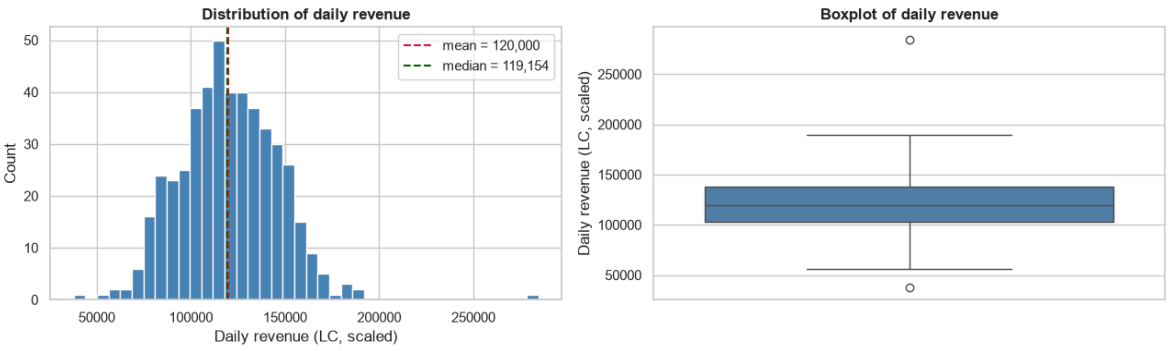
```
count       470
mean    120,000
std      26,072
min      37,733
5%       80,102
25%     102,720
50%     119,154
75%     138,021
95%     160,080
max     284,489
Name: revenue_scaled, dtype: float64
```

In [5]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].hist(df["revenue_scaled"], bins=40, color="steelblue", edgecolor="white")
axes[0].axvline(
    df["revenue_scaled"].mean(),
    color="crimson",
    linestyle="--",
    label=f"mean = {df['revenue_scaled'].mean():,.0f}",
)
axes[0].axvline(
    df["revenue_scaled"].median(),
    color="darkgreen",
    linestyle="--",
    label=f"median = {df['revenue_scaled'].median():,.0f}",
)
axes[0].set_xlabel("Daily revenue (LC, scaled)")
axes[0].set_ylabel("Count")
axes[0].set_title("Distribution of daily revenue")
axes[0].legend()

sns.boxplot(y=df["revenue_scaled"], ax=axes[1], color="steelblue")
axes[1].set_ylabel("Daily revenue (LC, scaled)")
axes[1].set_title("Boxplot of daily revenue")

plt.tight_layout()
plt.savefig("../charts/nb01_distribution.png", dpi=120, bbox_inches="tight")
plt.show()
```



Top and bottom three days

Attributed causes for the six sample extremes.

In [6]:
```
CAUSES = {
    "2019-12-25": (
        "Institutional customer settling about a year of accumulated cre
dit "
        "purchases in one payment; a recording artefact, not a single-da
y sales peak."
    ),
    "2020-02-12": "Cash purchase of a high-cost prescription by an unins
ured patient.",
    "2020-01-05": "Cash purchase of a high-cost prescription by an unins
ured patient.",
    "2020-05-15": (
        "Friday during the spring 2020 COVID trough; pharmacy on reduced
-hour operation."
    ),
    "2020-05-08": (
        "Friday during the spring 2020 COVID trough; pharmacy on reduced
-hour operation."
    ),
    "2020-07-31": (
        "Friday and Eid al-Adha (public holiday); pharmacy on reduced-ho
ur operation."
    ),
}


def build_extremes(days):
    rows = []
    for _, r in days.iterrows():
        date_str = r["date"].strftime("%Y-%m-%d")
        rows.append(
            {
                "Date": date_str,
                "Day": r["date"].day_name(),
                "Revenue (LC)": f"{r['revenue_scaled']:,.0f}",
                "Attributed cause": CAUSES.get(date_str, "Not identifie
d"),
            }
        )
    return pd.DataFrame(rows)


def show_extremes(days, title):
    print(title)
    styled = (
        build_extremes(days)
        .style.hide(axis="index")
        .set_properties(**{"text-align": "left"})
        .set_properties(subset=["Date"], **{"white-space": "nowrap"})
        .set_table_styles([{"selector": "th", "props": [("text-align",
"left")]}])
    )
    display(styled)
```

```
show_extremes(df.nlargest(3, "revenue_scaled"), "Highest three revenue d
ays:")
print()
show_extremes(df.nsmallest(3, "revenue_scaled"), "Lowest three revenue d
ays:")
```

```
Highest three revenue days:
```

Date	Day	Revenue (LC)	Attributed cause
2019-12-25	Wednesday	284,489	Institutional customer settling about a year of accumulated credit purchases in one payment; a recording artefact, not a single-day sales peak.
2020-02-12	Wednesday	189,844	Cash purchase of a high-cost prescription by an uninsured patient.
2020-01-05	Sunday	187,490	Cash purchase of a high-cost prescription by an uninsured patient.

```
Lowest three revenue days:
```

Date	Day	Revenue (LC)	Attributed cause
2020-05-15	Friday	37,733	Friday during the spring 2020 COVID trough; pharmacy on reduced-hour operation.
2020-05-08	Friday	55,596	Friday during the spring 2020 COVID trough; pharmacy on reduced-hour operation.
2020-07-31	Friday	57,812	Friday and Eid al-Adha (public holiday); pharmacy on reduced-hour operation.

Skewness and excess kurtosis of daily revenue, on the full sample and excluding 25 December 2019:

In [7]:
```
print("Full sample:")
print(f"  Skewness:         {skew(df['revenue_scaled']):.3f}")
print(f"  Excess kurtosis:  {kurtosis(df['revenue_scaled']):.3f}")

df_ex = df[df["date"] != pd.Timestamp("2019-12-25")]
print("\nExcluding 25 December 2019:")
print(f"  Skewness:         {skew(df_ex['revenue_scaled']):.3f}")
print(f"  Excess kurtosis:  {kurtosis(df_ex['revenue_scaled']):.3f}")
```

```
Full sample:
  Skewness:         0.500
  Excess kurtosis:  2.692

Excluding 25 December 2019:
  Skewness:         0.002
  Excess kurtosis:  -0.252
```

On the full sample the distribution is mildly right-skewed (0.50) with heavier tails than a normal distribution (excess kurtosis 2.69). Almost all of that shape comes from 25 December 2019, about 6.3 standard deviations above the mean. Excluding that day, skewness is 0.002 and excess kurtosis −0.25: symmetric, with slightly thinner tails than a normal distribution.

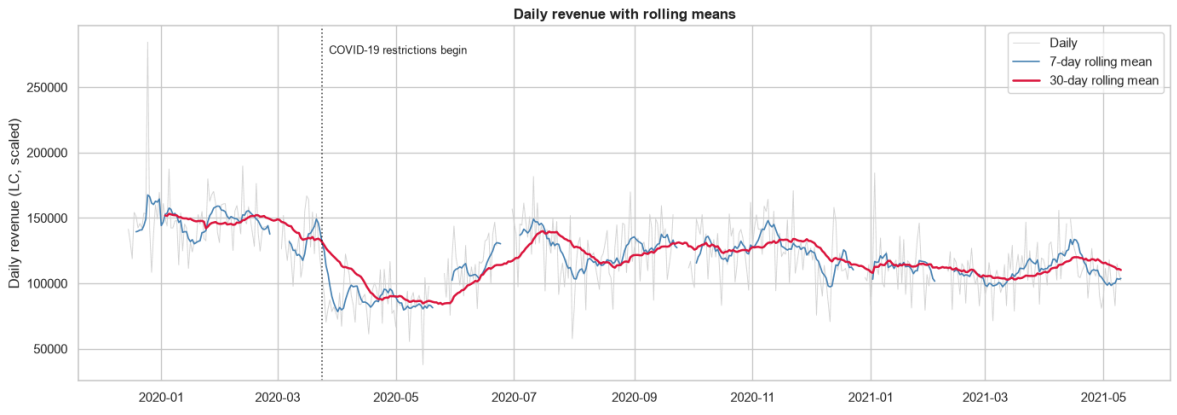
3. Time series and rolling means

In [8]:
```
# Daily frequency with NaN on missing days, so rolling windows count cal
endar days.
ts = df.set_index("date").asfreq("D")
ts["r7"] = ts["revenue_scaled"].rolling(7, min_periods=5).mean()
ts["r30"] = ts["revenue_scaled"].rolling(30, min_periods=20).mean()

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(ts.index, ts["revenue_scaled"], color="lightgrey", linewidth=0.
6, label="Daily")
ax.plot(
    ts.index, ts["r7"], color="steelblue", linewidth=1.2, label="7-day r
olling mean"
)
ax.plot(
    ts.index, ts["r30"], color="crimson", linewidth=1.8, label="30-day r
olling mean"
)

# Reference marker: 24 March 2020 (COVID-19 restrictions begin).
ax.axvline(
    pd.Timestamp("2020-03-24"), color="black", linestyle=":", linewidth=
1.2, alpha=0.7
)
ax.text(
    pd.Timestamp("2020-03-24"),
    ax.get_ylim()[1] * 0.95,
    "  COVID-19 restrictions begin",
    fontsize=9,
    va="top",
)

ax.set_ylabel("Daily revenue (LC, scaled)")
ax.set_title("Daily revenue with rolling means")
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("../charts/nb01_timeseries.png", dpi=120, bbox_inches="tigh
t")
plt.show()
```



In [9]:
```
# Pre-restrictions decline: 3–23 March 2020 mean vs January–February 202
0 mean.
janfeb = ts.loc["2020-01-01":"2020-02-29", "revenue_scaled"].mean()
early_march = ts.loc["2020-03-03":"2020-03-23", "revenue_scaled"].mean()
```

```
print(f"January–February 2020 mean: {janfeb:,.0f}")
print(f"3–23 March 2020 (pre-restrictions) mean: {early_march:,.0f}")
print(
    f"Early March 2020 vs January–February 2020: {(early_march / janfeb
- 1) * 100:+.1f}%"
)
```

```
January–February 2020 mean: 147,009
3–23 March 2020 (pre-restrictions) mean: 132,278
Early March 2020 vs January–February 2020: -10.0%
```

The rolling means include 25 December 2019, which lifts them in late December 2019; the spike is the recording artefact described in section 2. The 30-day rolling mean shifts downward beginning in early March 2020, ahead of the 24 March restrictions marker. Mean revenue over 3 to 23 March 2020 was about 10% below the January–February 2020 average, so the decline started before restrictions began. The series recovers partially through mid-2020 and drifts lower over the remainder of the sample. The pattern is compatible with a structural change and motivates the formal break-date analysis in NB02.

4. Monthly aggregates

In [10]:
```
# Level comparisons (monthly means, day-of-week means, month-phase mean
s) set
# aside 25 December 2019, per the introduction. The full series (ts) is
kept
# for coverage, distribution, and the time-series chart, where the day a
ppears
# as the December 2019 spike.
levels = ts.copy()
levels.loc[pd.Timestamp("2019-12-25"), "revenue_scaled"] = np.nan
```

In [11]:
```
monthly = levels["revenue_scaled"].resample("ME").mean().dropna()


# Colour scheme: pre-restrictions (blue), transition month straddling
# 24 March 2020 (gold), COVID-period months (red). December 2019 and May
2021
# are partial months (* in the label).
def month_colour(d):
    if d < pd.Timestamp("2020-03-01"):
        return "steelblue"
    elif d < pd.Timestamp("2020-04-01"):
        return "goldenrod"
    else:
        return "crimson"


colours = [month_colour(d) for d in monthly.index]

partial_months = {"2019-12", "2021-05"}
labels = [
    f"{d.strftime('%Y-%m')}*"
    if d.strftime("%Y-%m") in partial_months
    else d.strftime("%Y-%m")
    for d in monthly.index
```

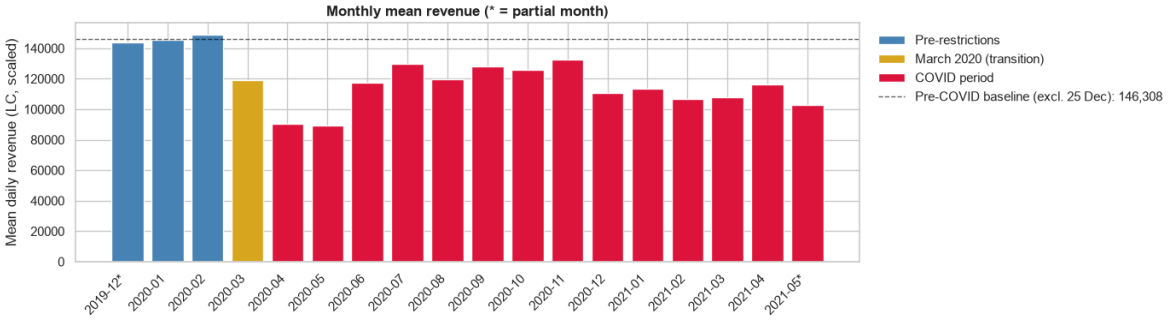
```
]

fig, ax = plt.subplots(figsize=(14, 4))
ax.bar(labels, monthly.values, color=colours, edgecolor="white")

# Baseline reference: mean of December 2019 to February 2020 in the leve
l-comparison series
# (which already excludes 25 December 2019 — a credit-settlement recordi
ng artefact).
baseline = levels.loc["2019-12":"2020-02", "revenue_scaled"].mean()
baseline_line = ax.axhline(
    baseline,
    color="black",
    linestyle="--",
    linewidth=1,
    alpha=0.6,
    label=f"Pre-COVID baseline (excl. 25 Dec): {baseline:,.0f}",
)

# Legend outside on the right so it does not sit over the bars.
colour_patches = [
    Patch(facecolor="steelblue", label="Pre-restrictions"),
    Patch(facecolor="goldenrod", label="March 2020 (transition)"),
    Patch(facecolor="crimson", label="COVID period"),
]
ax.legend(
    handles=colour_patches + [baseline_line],
    loc="upper left",
    bbox_to_anchor=(1.01, 1),
    frameon=False,
)

ax.set_ylabel("Mean daily revenue (LC, scaled)")
ax.set_title("Monthly mean revenue (* = partial month)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("../charts/nb01_monthly.png", dpi=120, bbox_inches="tight")
plt.show()
```



In [12]:

```
# 2021 persistence: mean daily revenue in Jan–May 2021 vs the pre-COVID
baseline
# already computed in the monthly-plot cell above (variable `baseline`).
year_2021 = levels.loc["2021-01":"2021-05", "revenue_scaled"].mean()
print(f"Pre-COVID baseline (Dec 2019–Feb 2020, excl. 25 Dec): {baselin
e:,.0f}")
print(f"Mean daily revenue, Jan–May 2021: {year_2021:,.0f}")
print(f"Jan–May 2021 vs pre-COVID baseline: {(year_2021 / baseline - 1)
* 100:+.1f}%")
```

```
Pre-COVID baseline (Dec 2019–Feb 2020, excl. 25 Dec): 146,308
Mean daily revenue, Jan–May 2021: 110,639
Jan–May 2021 vs pre-COVID baseline: -24.4%
```

5. Day-of-week pattern

In [13]:
```python
# Local business week: Sunday to Thursday; weekend: Friday and Saturday.
LABEL_ORDER = [
    "Sunday",
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
]

# `levels` already excludes 25 December 2019 for level comparisons.
work = levels.dropna(subset=["revenue_scaled"]).copy()
work["day_name"] = work.index.day_name()

dow_stats = (
    work.groupby("day_name")["revenue_scaled"]
    .agg(["mean", "std", "count"])
    .reindex(LABEL_ORDER)
)
print(dow_stats)
```

```
                mean     std  count
day_name
Sunday     126,824 24,529     68
Monday     123,164 23,437     68
Tuesday    122,385 22,563     66
Wednesday  121,409 23,661     66
Thursday   122,901 22,355     67
Friday      95,589 22,511     67
Saturday   125,181 22,045     67
```

In [14]:
```python
fig, ax = plt.subplots(figsize=(9, 4))

# Business days blue (Sunday to Thursday); weekend days salmon (Friday, Saturday).
colours = ["steelblue"] * 5 + ["lightsalmon"] * 2

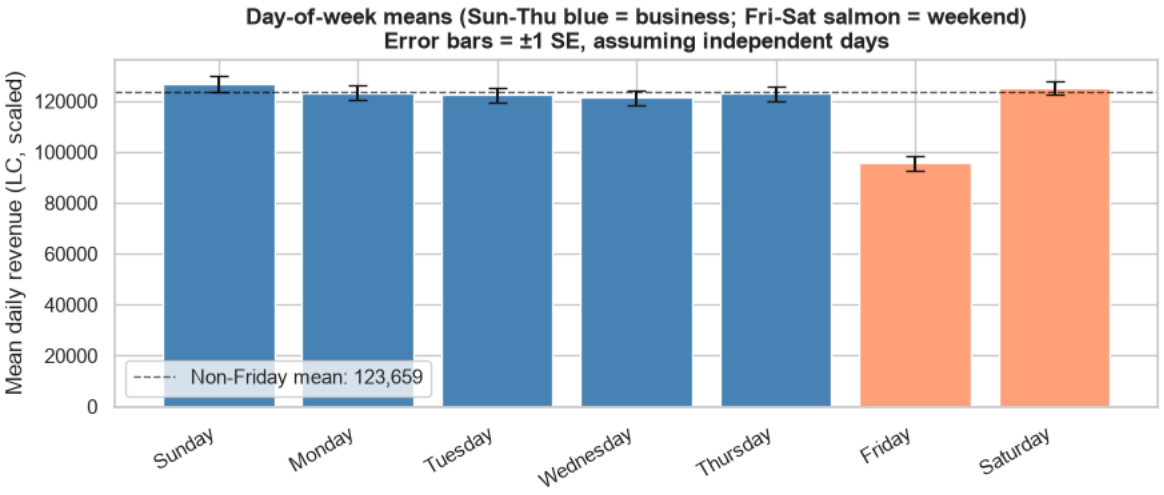
# Non-Friday reference: weighted mean of the six non-Friday days.
# Using this as the baseline isolates the Friday effect rather than letting
# it drag the reference down for every other day.
non_fri = dow_stats.drop("Friday")
non_friday_ref = (non_fri["mean"] * non_fri["count"]).sum() / non_fri["count"].sum()

ax.bar(
    dow_stats.index,
    dow_stats["mean"],
    yerr=dow_stats["std"] / np.sqrt(dow_stats["count"]),
    capsize=5,
    color=colours,
    edgecolor="white",
```

```
)
ax.axhline(
    non_friday_ref,
    color="black",
    linestyle="--",
    linewidth=1,
    alpha=0.6,
    label=f"Non-Friday mean: {non_friday_ref:,.0f}",
)
ax.set_ylabel("Mean daily revenue (LC, scaled)")
ax.set_title(
    "Day-of-week means (Sun-Thu blue = business; Fri-Sat salmon = weeken
d)\n"
    "Error bars = ±1 SE, assuming independent days"
)
ax.legend()
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig("../charts/nb01_dow.png", dpi=120, bbox_inches="tight")
plt.show()

print("\nEach day vs the mean of the six non-Friday days:")
for day in LABEL_ORDER:
    delta_pct = (dow_stats.loc[day, "mean"] / non_friday_ref - 1) * 100
    print(f"  {day:<10} {delta_pct:+6.1f}%")

# Bound the demand-side share of the Friday effect: drop the lowest 20%
of Fridays.
# The mean of the top 80% of Fridays is at least as high as the mean of
any subset
# of size >= 80% (including the normal-hours Fridays), provided no more
than about
# 20% of Fridays ran on reduced hours. The remaining gap vs the non-Frid
ay mean is
# therefore a lower bound on the Friday effect for normal-hours days.
fri = work.loc[work["day_name"] == "Friday", "revenue_scaled"]
upper = fri[fri > fri.quantile(0.20)]
lower_bound_pct = (upper.mean() / non_friday_ref - 1) * 100
print(f"\nFriday with lowest 20% removed: {lower_bound_pct:+.1f}% vs non
-Friday mean")
print("  (lower bound on the Friday effect for normal-hours days)")
```



```
Each day vs the mean of the six non-Friday days:
  Sunday       +2.6%
  Monday       -0.4%
  Tuesday      -1.0%
  Wednesday    -1.8%
  Thursday     -0.6%
  Friday      -22.7%
  Saturday     +1.2%

Friday with lowest 20% removed: -16.6% vs non-Friday mean
  (lower bound on the Friday effect for normal-hours days)
```

Friday is the only day that stands out: about 23% below the mean of the other six days. Against that same reference, every other day is within about 3% (Sunday +2.6%; the rest between –1.8% and +1.2%). Removing the lowest 20% of Fridays leaves a mean at least as high as that of the normal-hours Fridays, whichever days those were, provided no more than about 20% of Fridays ran on reduced hours (a threshold consistent with the pharmacy's records for this period). The remaining gap, about 17%, is therefore a lower bound on the Friday effect on normal-hours days, and reduced hours account for at most about 6 of the 23 percentage points. NB04 tests the pattern with HAC standard errors and controls for regimes, Ramadan and Eid, and shows the Friday dip deepened after March 2020, from about –13% before COVID to about –31% in the acute crash and about –22% on the lower plateau (measured against the non-Friday mean within each regime); opening hours did not change materially, so the deepening reflects demand, most plausibly Friday's non-prescription product mix, which the COVID-19 restrictions curtailed most.

6. Month-phase quick look

In [15]:

```python
def month_phase(d):
    days_in_month = d.days_in_month
    if d.day <= 5:
        return "start (day 1–5)"
    elif d.day > days_in_month - 5:
        return "end (last 5)"
    else:
        return "middle"

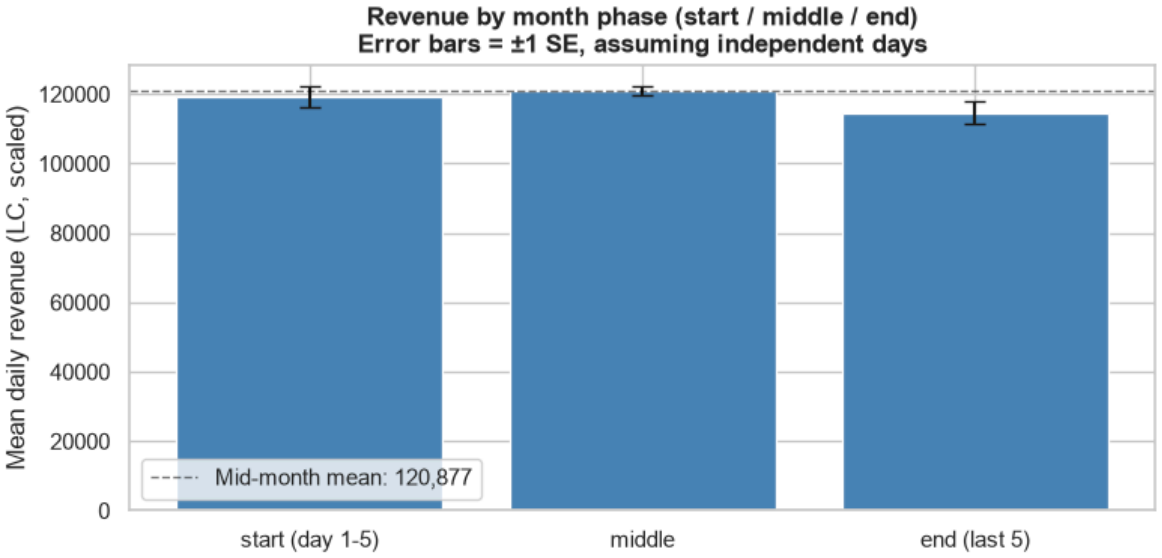

work["month_phase"] = work.index.map(month_phase)
phase_stats = (
    work.groupby("month_phase")["revenue_scaled"]
    .agg(["mean", "std", "count"])
    .reindex(["start (day 1–5)", "middle", "end (last 5)"])
)
print(phase_stats)

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(
    phase_stats.index,
    phase_stats["mean"],
    yerr=phase_stats["std"] / np.sqrt(phase_stats["count"]),
    capsize=5,
    color="steelblue",
```

```
    edgecolor="white",
)
mid_mean = phase_stats.loc["middle", "mean"]
ax.axhline(
    mid_mean,
    color="black",
    linestyle="--",
    linewidth=1,
    alpha=0.5,
    label=f"Mid-month mean: {mid_mean:,.0f}",
)
ax.set_ylabel("Mean daily revenue (LC, scaled)")
ax.set_title(
    "Revenue by month phase (start / middle / end)\n"
    "Error bars = ±1 SE, assuming independent days"
)
ax.legend()
plt.tight_layout()
plt.savefig("../charts/nb01_month_phase.png", dpi=120, bbox_inches="tigh
t")
plt.show()

start_pct = (phase_stats.loc["start (day 1-5)", "mean"] / mid_mean - 1)
* 100
end_pct = (phase_stats.loc["end (last 5)", "mean"] / mid_mean - 1) * 100
print(f"\nStart-of-month vs mid: {start_pct:+.1f}%")
print(f"End-of-month vs mid:   {end_pct:+.1f}%")
```

```
                    mean    std  count
month_phase
start (day 1-5) 119,236 27,051     79
middle          120,877 23,916    319
end (last 5)    114,591 26,835     71
```




```
Start-of-month vs mid: -1.4%
End-of-month vs mid:   -5.2%
```

7. Summary and next steps

Scope caveat. All findings describe counter cash receipts, about 65% of total revenue; see the introduction for the recording basis.

Patterns and open questions from this EDA that motivate the formal tests in the follow-on notebooks:

1. **Level shift from March 2020.** Visible in the 30-day rolling mean, though the visual signal alone is not decisive. The rolling mean starts drifting down before the 24 March restrictions marker: the period 3 to 23 March 2020 is already about 10% below the January–February 2020 average. Tested in NB02 with a sup-Wald scan, PELT and a block bootstrap on the number of breaks, where the break dates are estimated with intervals.
2. **Post-break persistence.** Revenue in January–May 2021 averaged about 24% below the pre-COVID (December 2019 to February 2020, excluding the 25 December 2019 one-off) baseline, consistent with a persistent shift rather than a temporary excursion. The persistence question is examined in NB02.
3. **Variance across regimes.** Whether within-regime variance differs across regimes is not decidable from the EDA visuals alone (the boxplot pools all months). Tested in NB03.
4. **Friday effect.** Friday means sit about 23% below the mean of the other six days, while Saturday and the working days Sunday–Thursday all track a common working-day level within about 3%. Removing the lowest 20% of Fridays leaves at least about 17% below the non-Friday reference, so at most about 6 of the 23 percentage points are attributable to reduced-hours days (given no more than about 20% of Fridays ran on reduced hours). Tested with HAC standard errors in NB04.
5. **Month-phase pattern.** No start-of-month lift is visible (about –1.4% versus mid-month, descriptive), and the last five days of the month are about 5.2% below mid-month. NB04 tests whether this follows customers' income timing (salaries, pensions), which acts on counter receipts in two ways: purchases fall as money runs short before payday, and customers who buy on account settle after it. I do not claim to identify the mechanism.